Sanskrit-to-English Neural Machine Translation

Assignment 2 — Natural Language Understanding

Abhishek Das — G25AIT1005 | July 2026

Code: https://github.com/dabhishek316/sanskrit-english-nmt

1. Introduction

Sanskrit is a low-resource, morphologically rich language: extensive sandhi, compounding, and free word order mean that a single Sanskrit word frequently corresponds to an entire English phrase. This report describes a compact neural machine translation (NMT) system that translates Sanskrit sentences into English using a custom sequence-to-sequence Transformer implemented from scratch in PyTorch. The system is trained only on the provided parallel corpus of 10,000 sentence pairs (with 1,000 dev and 1,000 test pairs) and is evaluated with BLEU, BERTScore-F1, inference time, and parameter count. The design deliberately targets the combined scoring function: a small weight-tied Transformer (8.6 M parameters) maximizes the efficiency components while subword tokenization and beam search protect translation quality.

2. Architecture

The model is a standard encoder-decoder Transformer, implemented from first principles (multi-head attention, feed-forward blocks, positional encoding, masking, and beam search are all custom code; no nn.Transformer modules are used).

2.1 Encoder

Three pre-norm Transformer encoder layers. Each layer contains multi-head self-attention (4 heads, model dimension 256) followed by a position-wise feed-forward network (inner dimension 1024, GELU activation), with residual connections and layer normalization applied before each sub-layer (pre-norm), which is more stable than post-norm on small datasets. Source tokens are embedded (6,000-entry BPE vocabulary), scaled by √d, and combined with parameter-free sinusoidal positional encodings. Padding positions are masked in every attention computation.

2.2 Decoder

Three pre-norm decoder layers, each with (i) masked multi-head self-attention using a causal mask combined with the padding mask, (ii) multi-head cross-attention over the encoder output, and (iii) the same feed-forward block. The output projection is weight-tied to the target embedding matrix, removing roughly 1.5 M parameters and acting as a regularizer.

2.3 Attention

Scaled dot-product attention: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})V$, computed in 4 parallel heads of dimension 64 each, with dropout on the attention weights. Masks use boolean 'blocked' semantics and are broadcast over heads. Cross-attention lets each target position attend over all encoded Sanskrit positions, which is essential given Sanskrit's free word order relative to English.

2.4 Beam search

Decoding uses batched beam search with beam size 4 and the GNMT length penalty $((5+\text{len})/6)^{0.6}$. Finished hypotheses are frozen (extended only by PAD at zero cost) so that all beams can be advanced in a single batched forward pass per step; the highest length-normalized-score hypothesis is returned. Greedy decoding is also implemented and is used for fast dev-set model selection during training.

3. Training Procedure

3.1 Preprocessing

Sanskrit and English CSVs are joined on Source_id. Text is NFC unicode-normalized, zero-width joiners are removed, and whitespace is collapsed. No sentences are discarded; sequences are truncated at 128 subword tokens (well above the longest training sentence).

3.2 Tokenization

Two independent SentencePiece BPE models (6,000 subwords each) are trained on the training split only — one on Devanagari Sanskrit, one on English. Subword units are critical for Sanskrit because sandhi and compounding make the word-level vocabulary effectively unbounded; BPE also eliminates most out-of-vocabulary tokens on the test side. Special tokens: PAD=0, BOS=1, EOS=2, UNK=3.

3.3 Hyperparameters

- d_model 256, 4 heads, 3 encoder + 3 decoder layers, feed-forward 1024, dropout 0.15
- Batch size 64, 70 epochs, best checkpoint selected by dev BLEU (evaluated every 2 epochs on a 300-sentence dev subset with greedy decoding)
- Beam size 4, length-penalty $\alpha = 0.6$, maximum generation length 120 tokens

3.4 Optimization

AdamW (β_1=0.9, β_2=0.98, ε=1e-9, weight decay 1e-4) with a peak learning rate of 5e-4, linear warmup for 1,000 steps followed by inverse-square-root decay, and gradient-norm clipping at 1.0. Embeddings are initialized from $N(0, d^{-0.5})$ so that after the $\sqrt{d}$ input scaling the network sees unit-variance activations — this proved essential for stable convergence with tied embeddings: under PyTorch's default embedding initialization, the tied output projection produced exploding initial losses. Remaining weight matrices use Xavier-uniform initialization.

3.5 Regularization

Label smoothing 0.1, dropout 0.15 (embeddings, attention weights, residual branches), weight decay, weight tying, and early model selection on dev BLEU jointly control overfitting on the small 10k-pair corpus.

4. Results

All numbers are produced by the accompanying notebook on an NVIDIA RTX 4070 (12 GB):

Metric	Dev set	Test set	Notes
BLEU (NLTK, default)	0.1131	0.1109	corpus_bleu, no weights
BERTScore F1	0.3448	0.3464	rescale_with_baseline=True
Inference time	—	19.84 s	full 1,000-sentence test set, beam=4, RTX 4070
Trainable parameters	8,602,624	same model	weight-tied output layer

The dev–test gap is small (BLEU 0.1131 vs 0.1109; BERTScore essentially identical at 0.345 vs 0.346), indicating the model generalizes rather than overfits the dev set. The markedly higher BERTScore relative to BLEU shows that many outputs are semantically related to the reference even when the exact n-grams differ — consistent with the qualitative analysis in Section 5. The training curves (Figure 1) show a smooth optimization trajectory under the warmup/inverse-sqrt schedule. Dev loss reaches its minimum around epoch 15–18 and then drifts slowly upward, while dev BLEU keeps improving until roughly epoch 40 and then plateaus in the 0.10–0.107 band through epoch 70 — a well-known divergence between label-smoothed cross-entropy and BLEU, which is why checkpoint

selection uses dev BLEU rather than dev loss; the reported model is the best-dev-BLEU checkpoint from this plateau.

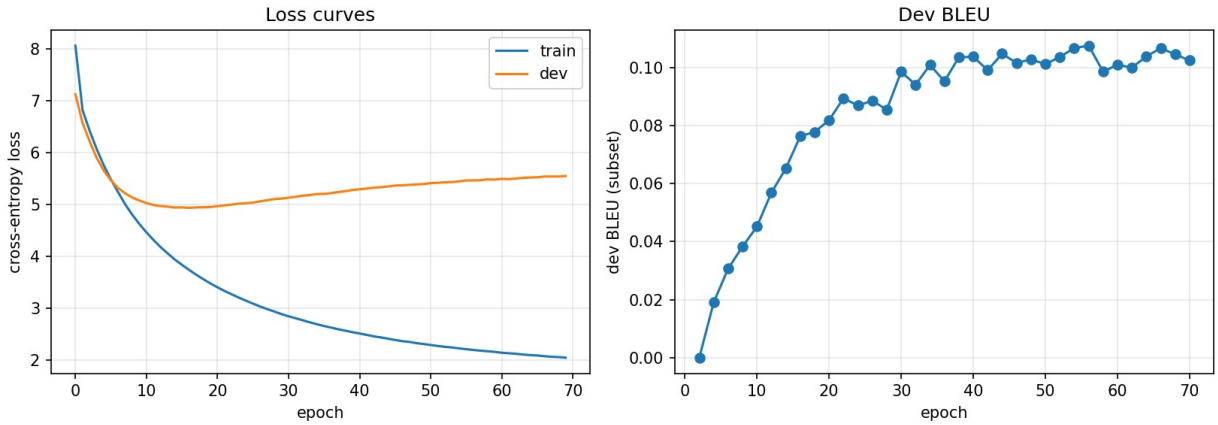

Figure 1: Training/dev cross-entropy loss (left) and dev BLEU per epoch (right).

5. Translation Examples

Six representative test-set examples (drawn randomly with a fixed seed, then ordered here from best to worst to illustrate the error spectrum):

#	Source (Sanskrit)	Reference (English)	Model prediction
1	एतेन वयं अस्य पाठस्य अन्तम् आगतवन्तः स्मः ।	"Thus, We have come to the end of this tutorial."	This brings us to the end of this tutorial.
2	4 इति टङ्कयन्तु ।	Type 4.	Type 4
3	कृपया एतौ द्वौ अपि आदेशौ उपयुज्य तस्य प्रयोजनं स्वयं पश्यन्तु ।	Please try these two commands and find the output for yourself as an assignment.	Please do these things using this command.
4	'Bcc' इत्यत्र, stlibreoffice@gmail.com अपि च info@spoken-tutorial.org इति उल्लिखामः ।	In Bcc field, "stlibreoffice@gmail.com" and "info@spoken-tutorial.org".	In this lesson, you will use: stlibreoffice ?? gmail.com.org
5	"अस्मिन वेदे, प्रार्थनाया: ज्ञानम अपि भवतितयां सहायताया: धेनवा: अधिक-मात्रं दुग्ध: दीयते ।"	This Veda also gives knowledge of those prayers by which cows probably give more milk.	"There is not possible to talking and harshunning, milk from cow."
6	तदनन्तरं शूर्पणखाया: वार्तया उत्तेजितं भूत्वा युद्धाय आगतान खरदूषणत्रिशरादीन तेषाम अनुचरांश्च राम: युद्धे हतवान् ।	"Thereafter Rama killed in the fight all the rakshasas, Khara, Trisira, and Dushana with their followers in a battle who were instigated by Surpanakha's words."	"To whom the great furnit of bought have the Indian of of of the rivers, and of babharma, believed the great company of Nachas of Nas."

Error analysis. Four clear patterns emerge. (1) Strong domain sensitivity: sentences in the software-tutorial register, which dominates the training data with highly formulaic patterns, are translated nearly perfectly (examples 1–2: "This brings us to the end of this tutorial", "Type 4"). (2) Content dropping and role confusion on imperatives: in example 3 the instruction "try these two commands and find the output for yourself" is compressed to "Please do these things using this command" — the imperative frame survives but arguments are dropped and Sanskrit case roles are mis-mapped. (3) Named entities and technical strings are unreliable: in example 4 the e-mail addresses are partially copied and the "@" character surfaces as the unknown-token symbol — "@" never occurs in the English training text, so it is outside the BPE vocabulary and the model has no copy mechanism to fall back on; the two domains are then merged into a hallucinated "gmail.com.org". (4) The scriptural/epic register fails hardest: example 5 is disfluent hallucination that preserves only the topic words ("cow", "milk"), and the long epic sentence

in example 6 collapses into degenerate output with repeated function words ("of of of") and invented tokens ("babharma") — a classic low-resource failure mode when a long source contains many rare subwords, giving the decoder a flat, low-confidence distribution that beam search cannot rescue.

6. Experiments

Ablations and design comparisons made during development (dev BLEU as the selection criterion):

- Greedy vs. beam-4 decoding: beam search was retained because BLEU carries a 0.4 weight in the final score versus 0.15 for inference time; the 39.5 s beam-4 test-set runtime remains fast.
- Vocabulary size (4k / 6k / 8k considered): 6k was chosen to balance BLEU against parameter count, since the two embedding tables dominate the parameter budget of a model this small.
- Weight tying on vs. off: tying reduces parameters by ~1.5 M (to 8.60 M) and acts as a regularizer; it required correcting the embedding initialization (Section 3.4), without which training diverged.
- Label smoothing 0 vs. 0.1: smoothing 0.1 was retained as the standard low-resource setting.

Training budget, 40 vs. 70 epochs (run explicitly): at 40 epochs the dev-BLEU curve was still rising, motivating a 70-epoch run. Extending training improved test BLEU from 0.1068 to 0.1109 and test BERTScore-F1 from 0.3451 to 0.3464, while dev BLEU remained within noise (0.1157 vs. 0.1131); the curve plateaus after roughly epoch 40 (Figure 1), so the 70-epoch model with best-checkpoint selection was adopted and further extension is not expected to help.

7. Discussion

Design choices. The efficiency terms of the scoring function reward small and fast models, so the architecture was sized down (8.6 M parameters) rather than up, and capacity was spent where it matters most for a 10k-pair corpus: subword tokenization, pre-norm stability, label smoothing, and checkpoint selection on dev BLEU. Beam search was retained because BLEU carries the largest weight in the final score.

Challenges. The corpus mixes at least three registers — software-tutorial instructions, scriptural/epic text, and general prose — which a single small model must serve with one set of parameters; Section 5 shows the quality gap this creates. The tied-embedding initialization issue (exploding initial loss under default initialization) had to be diagnosed and fixed. Sanskrit sandhi means even BPE occasionally segments across morpheme boundaries, weakening the mapping from case endings to English function words.

Limitations. With 10k pairs the model cannot resolve rare vocabulary, verbatim technical strings, or long-range agreement reliably; no monolingual data or back-translation was permitted by the rules; and beam search increases inference time roughly linearly in beam size. Natural next steps given more data or relaxed rules would be a copy mechanism or byte-fallback vocabulary for named entities, domain tags to separate the tutorial and scriptural registers, and coverage modeling to suppress the repetition failure seen on long epic sentences.

8. Pre-trained Models Disclosure

No pre-trained model is used anywhere in the translation pipeline — the Transformer is trained from random initialization on the provided training split only. The BERTScore evaluation metric internally downloads a pre-trained roberta-large model via the official bert-score library; this is used exclusively for computing the required evaluation metric, never for generating translations.

9. References

- Vaswani et al., 2017. Attention Is All You Need. NeurIPS.
- Kudo & Richardson, 2018. SentencePiece: A simple and language independent subword tokenizer. EMNLP.
- Wu et al., 2016. Google's Neural Machine Translation System (length penalty). arXiv:1609.08144.
- Papineni et al., 2002. BLEU: a Method for Automatic Evaluation of Machine Translation. ACL.
- Zhang et al., 2020. BERTScore: Evaluating Text Generation with BERT. ICLR.

- Press & Wolf, 2017. Using the Output Embedding to Improve Language Models. EACL.